Open Source Formal Verification

Parametric design for formal

Yann Thoma

Reconfigurable and Embedded Digital Systems Institute
Haute Ecole d'Ingénierie et de Gestion du Canton de Vaud



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License

November 2026

Introduction

- Issues of formal verification:
  - Time
    - Related to the depth of search
  - Space
    - The bigger the data, the longer the formal verification
- How can we reduce verification runtime and memory use without losing the behaviors we need to check?

Reducing Space

- A system composed of n D flip flops has potentially 2^n states

$\Rightarrow$ Each D flip flop potentially multiplies the state space by 2

$\Rightarrow$ Reducing the number of D flip flops reduces the space

- Space as memory
  - Reduce memories sizes
  - Reduce FIFOs sizes
- Space as data representation
  - Can the design be verified with a smaller data size?
    - For instance, a FIFO can be verified with 1-bit data
  - Arithmetic computations do not necessarily need 32-bit data to check its behavior

Abstraction: reduce carefully

- Stored data bits contribute to the design's state: an N-bit register has up to 2^N encodings (the reachable set may be smaller)
- Reducing a FIFO's data width from 32 bits to 1 bit reduces each stored word from 32 state bits to 1; this can make control properties such as full/empty handling and transaction ordering cheaper to check
- The reduced model is useful only if it preserves the behavior relevant to the property
  - A 1-bit FIFO check does not exercise interactions among payload bits or width-specific truncation and arithmetic
  - Choose widths and parameter values that retain the boundary behavior the property depends on, and verify width-specific behavior at representative or production configurations too

Reducing time

- How to rapidly reach interesting moments?
- Reduce time by any means
  - Counter maximal value
  - Sampling frequency
  - Data rate
- Interesting corner cases: Counter overflow
  - Does the counter wrap from its maximum value to zero?
    - Is it supposed to happen?
    - Is it handled correctly?

How to reduce?

1. Value as an input port
   - Use `assume` to fix the value for formal verification
2. Value as a generic parameter
   - Formal verification with small values
3. Value as a constant in a package
   - Use another version of the package for formal verification

Example: Counter

Bad

```
entity counter is
port (
  clk_i, rst_i : in  std_logic;
  enable_i     : in  std_logic;
  value_o      : out std_logic_vector(31 downto 0)
);
```

Good

```
entity counter is
generic (SIZE: integer := 32);
port (
  clk_i, rst_i : in  std_logic;
  enable_i     : in  std_logic;
  value_o      : out std_logic_vector(SIZE-1 downto 0)
);
```

Example: UART sender-receiver

- Baud rate as an input, generic parameter or constant in a package
- Size of the FIFO : idem
- Size of the data to be sent : idem

Package

Synthesis

```
package mydesign_pkg is

    constant COUNTER_SIZE : integer := 32;

    constant DATA_SIZE : integer := 16;

    constant FIFO_SIZE : integer := 256;

    subtype data_t is integer
        range 0 to 2**DATASIZE - 1;

    -- 9600 at 50MHz
    constant CLK_PER_BIT : integer := 5208;

end package;
```

Verification

```
package mydesign_pkg is

    constant COUNTER_SIZE : integer := 8;

    constant DATA_SIZE : integer := 4;

    constant FIFO_SIZE : integer := 4;

    subtype data_t is integer
        range 0 to 2**DATASIZE - 1;


    constant CLK_PER_BIT : integer := 8;

end package;
```

What values for generics?

- Impossible to test all values for generics
  - Especially if you have more than one generic parameter
- Then, how to choose the relevant ones?
- It depends on the meaning of the generics
  - Related to space
  - Related to time

What values for generics? - Space

- When generics represent
  - Size of registers
  - Number of elements in a memory
  - Size of the memory elements
  - Data values for mathematical computation
  - ...
- Choose
  - Small, legal values that still exercise the behavior
    - For FIFO depth, compare 3 and 4 (and 1 if supported)
    - For data width, use the minimum legal width plus a wider case
  - Include non-power-of-two sizes, such as depth 3, to expose assumptions that pointers wrap naturally at a power-of-two boundary

What values for generics? - Time

- When generics represent
  - Max value of a counter
  - Timing values of a PWM
  - Number of clock cycles per bit for a serial transmitter
  - Minimum time between two events
  - Maximum time between two events
  - ...
- Choose
  - Short values that preserve the key timing cases
    - For a counter terminal count, try 1, 3, and 4: check the minimum case, a non-power-of-two wrap, and a power-of-two boundary
    - For a PWM, include period 3 or 5 and boundary duty values (0, 1, one less than the period, and the period when legal)
  - Check the value's documented minimum/maximum and the cycle where wrap, compare, or event timing occurs

Flow limitation: generic SERE bounds

- In the course flow (OSS CAD Suite snapshot 2026-10-14), a VHDL generic cannot be used directly as a SERE repetition bound

A generic bound is not accepted by this flow

```
assert always {req} |=> {(not ack)[*0 to MAX_WAIT];ack};
```

- Workaround: use a verification-package constant for the bound

Use a constant matching the configured generic value

```
assert always {req} |=> {(not ack)[*0 to CONST_MAX_WAIT];ack};
```

  - Keep the package constant consistent with the generic value passed to the formal run
  - Generate the package for each configuration when automating runs (see CI and scripting)

Takeaway message

Whenever possible, use generic parameters or constants declared in packages

Not only for formal, but also for simulation-based verification